

电驱通讯协议

版本	修改内容	修改人	日期
V0.1	第一版发布	向磊	2025-08-13
V0.2	新增轨迹控制参数配置指令	向磊	2025-08-19
V0.3	新增非广播包MIT控制指令	向磊	2025-08-25
v0.4	新增MIT范围配置参数读写指令	向磊	2025-08-28
v0.5	新增0xFF通用广播包 优化广播包描述	向磊	2025-09-09
v0.6	新增0x95，0x96设置和周期上报CVP 修改3.3.1 特殊协议包描述 对应软件版本v1.14之后	向磊	2025-12-02
v0.7	新增0x03，0x0c力矩读写设置 增加返回单位备注 对应软件版本v1.14之后	向磊	2025-12-10
v0.8	修复 0xFF 读取警告的改写冲突	向磊	2025-12-22

1. 概述

本文档详细描述了镜头电机控制系统的CAN通讯协议。该系统基于STM32微控制器，使用CAN总线进行通讯，支持多种控制模式和参数配置。

本文档所有数据相关单位：**位置信息**：弧度，**速度信息**：弧度/秒，**电流信息**：安培，**电压信息**：伏特，**温度信息**：摄氏度，**扭矩信息**：牛米

2. 协议格式

2.1 基本帧格式

标准CAN帧格式:

- ID (11位): 设备地址标识
- DLC: 数据长度码

- 广播包：Data[0-64]；标准包：Data[0-7]；数据字段

2.2 命令帧格式

1. 广播包命令帧：

命令帧由以下部分组成：

- Data[0]: 命令码 (CMD)
- Data[1-7]: 参数数据

2. 标准包命令帧

- Data[0-64]: 参数数据

 仅MIT控制支持广播包，故广播包在数据段内无CMD字段，只要是广播包ID，则默认按照MIT控制协议解析命令

3. 标准协议包

3.1 命令包格式

命令包格式如下图，发送八个字节，一个CMD和最多七个数据内容。

标准包



3.2 命令列表

基础命令

命令码	命令名称	方向	描述
0x00	HAND_SHAKE	R/W	握手命令
0x01	R_VERSION	R	读取版本信息

0x02	R_ADDR	R	读取地址信息
0x0D	SAVE_PARA	W	保存参数
0x3D	SET_ID	W	设置设备ID；ID设置范围1-128，其他ID号预留；
0x97	GO_TO_BOOTLOADER	W	进入引导程序模式

电机状态监控命令

命令码	命令名称	方向	数据格式	描述
0x03	R_TORQUE	R	float(4字节)	读取当前扭矩
0x04	R_CURRENT	R	float(4字节)	读取当前电流
0x05	R_VELOCITY	R	float(4字节)	读取当前速度
0x06	R_POSITION	R	float(4字节)	读取当前位置
0x2B	R_ON_OFF	R	uint8_t(1字节)	读取电机使能状态
0x45	R_VOLTAGE	R	float(4字节)	读取母线电压
0x55	R_CURRENT_MODE	R	uint8_t(1字节)	读取控制模式
0x5E	R_CORE_TEMP	R	float(4字节)	读取核心温度
0x5F	R_MOTOR_TEMP	R	float(4字节)	读取电机温度
0x60	R_INVERTER_TEMP	R	float(4字节)	读取逆变器温度
0x94	R_CVP	R	12字节	读取电流、速度、位置
0x95	S_CVP_FREQ	S	float(4字节)	设置CVP周期上报时间，单位MS，浮点数传输取整执行
0x96	R_CVP_FREQ	R	8字节	解析方式见3.3.1 数据上报，返回速度，位置，力矩和 错误码

电机控制命令

--	--	--	--	--

命令码	命令名称	方向	数据格式	描述
0x07	SET_MODE	W	uint8_t(1字节)	设置控制模式 代码块 <pre>1 typedef enum { 2 MODE_TORQUE = 0, 3 MODE_MIT = 1, 4 MODE_VELOCITY = 2, 5 6 MODE_PROFILE_VELOCITY 7 = 3, 8 MODE_POSITION = 4, 9 10 MODE_PROFILE_POSITION 11 = 5, 12 }MotorModeEnum;</pre>
0x08	SET_CURRENT	W	float(4字节)	设置电流（Amp），返回值见3.3.1特殊协议包返回值
0x09	SET_VELOCITY	W	float(4字节)	设置速度（rad/s），返回值见3.3.1特殊协议包返回值
0x0A	SET_POSITION	W	float(4字节)	设置位置（radian），返回值见3.3.1特殊协议包返回值
0x0B	SET_MIT	W	8字节	设置MIT，见3.3.3数据下发和返回
0x0C	SET_TOR	W	float(4字节)	设置扭矩（NM）
0x2A	S_ON_OFF	W	uint8_t(1字节)	设置电机使能(0x00=关闭,0x01=使能)

轨迹控制参数配置

命令码	命令名称	方向	数据格式	描述
0x1C	R_PT_V	R	float(4字节)	读取位置轨迹最大速度
0x1D	R_PT_A	R	float(4字节)	读取位置轨迹加速度
0x1E	R_PT_D	R	float(4字节)	读取位置轨迹减速度

0x1F	SET_PT_V	W	float(4字节)	设置位置轨迹最大速度
0x20	SET_PT_A	W	float(4字节)	设置位置轨迹加速度
0x21	SET_PT_D	W	float(4字节)	设置位置轨迹减速度
0x22	R_VT_V	R	float(4字节)	读取速度轨迹最大速度
0x23	R_VT_A	R	float(4字节)	读取速度轨迹加速度
0x24	R_VT_D	R	float(4字节)	读取速度轨迹减速度
0x25	SET_VT_V	W	float(4字节)	设置速度轨迹最大速度
0x26	SET_VT_A	W	float(4字节)	设置速度轨迹加速度
0x27	SET_VT_D	W	float(4字节)	设置速度轨迹减速度

PID参数配置命令

命令码	命令名称	方向	数据格式	描述
0x0E	SET_C_P	W	float(4字节)	设置电流环P参数
0x0F	SET_C_I	W	float(4字节)	设置电流环I参数
0x10	SET_V_P	W	float(4字节)	设置速度环P参数
0x11	SET_V_I	W	float(4字节)	设置速度环I参数
0x12	SET_P_P	W	float(4字节)	设置位置环P参数
0x13	SET_P_I	W	float(4字节)	设置位置环I参数
0x14	SET_P_D	W	float(4字节)	设置位置环D参数
0x15	R_C_P	R	float(4字节)	读取电流环P参数
0x16	R_C_I	R	float(4字节)	读取电流环I参数
0x17	R_V_P	R	float(4字节)	读取速度环P参数
0x18	R_V_I	R	float(4字节)	读取速度环I参数
0x19	R_P_P	R	float(4字节)	读取位置环P参数
0x1A	R_P_I	R	float(4字节)	读取位置环I参数
0x1B	R_P_D	R	float(4字节)	读取位置环D参数

0x2E	SET_C_PID_DL	W	float(4字节)	设置电流环PID输出限制
0x30	SET_V_PID_DL	W	float(4字节)	设置速度环PID输出限制
0x32	SET_P_PID_DL	W	float(4字节)	设置位置环PID输出限制
0x34	R_C_PID_DL	R	float(4字节)	读取电流环PID输出限制
0x36	R_V_PID_DL	R	float(4字节)	读取速度环PID输出限制
0x38	R_P_PID_DL	R	float(4字节)	读取位置环PID输出限制

输入限制命令

命令码	命令名称	方向	数据格式	描述
0x58	S_C_INPUT_LIMIT	W	float(4字节)	设置电流输入限制
0x59	R_C_INPUT_LIMIT	R	float(4字节)	读取电流输入限制
0x5A	S_V_INPUT_LIMIT	W	float(4字节)	设置速度输入限制
0x5B	R_V_INPUT_LIMIT	R	float(4字节)	读取速度输入限制
0x83	S_P_MAX_L	W	float(4字节)	设置位置上限
0x84	S_P_MIN_L	W	float(4字节)	设置位置下限
0x85	R_P_MAX_L	R	float(4字节)	读取位置上限
0x86	R_P_MIN_L	R	float(4字节)	读取位置下限

保护参数命令

命令码	命令名称	方向	数据格式	描述
0x61	S_INV_PROTECT_TEMP	W	float(4字节)	设置逆变器保护温度
0x62	R_INV_PROTECT_TEMP	R	float(4字节)	读取逆变器保护温度
0x6B	S_MOTOR_PROTECT_TEMP	W	float(4字节)	设置电机保护温度
0x6C	R_MOTOR_PROTECT_TEMP	R	float(4字节)	读取电机保护温度
0x70	S_CORE_PROTECT_TEMP	W	float(4字节)	设置核心保护温度
0x71	R_CORE_PROTECT_TEMP	R	float(4字节)	读取核心保护温度

0x72	S_UNDER_VOLT	W	float(4字节)	设置欠压保护值
0x73	R_UNDER_VOLT	R	float(4字节)	读取欠压保护值
0x74	S_OVER_VOLT	W	float(4字节)	设置过压保护值
0x75	R_OVER_VOLT	R	float(4字节)	读取过压保护值
0x76	S_OVER_VEL	W	float(4字节)	设置超速保护值
0x77	R_OVER_VEL	R	float(4字节)	读取超速保护值
0x78	S_OVER_CUR	W	float(4字节)	设置过流保护值
0x79	R_OVER_CUR	R	float(4字节)	读取过流保护值

电机参数命令

命令码	命令名称	方向	数据格式	描述
0x87	SET_HOME	W	-	设置原点
0xC0	S_MOTOR_L	W	float(4字节)	设置电机电感
0xC1	R_MOTOR_L	R	float(4字节)	读取电机电感
0xC2	S_MOTOR_R	W	float(4字节)	设置电机电阻
0xC3	R_MOTOR_R	R	float(4字节)	读取电机电阻
0xC4	S_GEAR_RATIO	W	float(4字节)	设置减速比
0xC5	R_GEAR_RATIO	R	float(4字节)	读取减速比
0xC7	S_CALI_START	W	-	开始校准
0xC8	S_CALI_CURRENT	W	float(4字节)	设置校准电流
0xC9	R_CALI_CURRENT	R	float(4字节)	读取校准电流
0xCA	S_TORQUE_COEFF	W	float(4字节)	设置转矩系数
0xCB	R_TORQUE_COEFF	R	float(4字节)	读取转矩系数

MIT范围参数命令

命令码	命令名称	方向	数据格式	描述
-----	------	----	------	----

0XA0	S_MIT_P_UPPER	W	float(4字节)	设置mit位置上限
0XA1	S_MIT_P_LOWER	W	float(4字节)	设置mit位置下限
0xA2	S_MIT_V_UPPER	W	float(4字节)	设置mit速度上限
0xA3	S_MIT_V_LOWER	W	float(4字节)	设置mit速度下限
0xA4	S_MIT_T_UPPER	W	float(4字节)	设置mit力矩上限
0xA5	S_MIT_T_LOWER	W	float(4字节)	设置mit力矩下限
0xA6	S_MIT_KP_RANGE	W	float(4字节)	设置mit Kp范围（0-范围值）
0xA7	S_MIT_KD_RANGE	W	float(4字节)	设置mit Kd范围（0-范围值）
0xA8	R_MIT_P_UPPER	R	float(4字节)	读取mit位置上限
0xA9	R_MIT_P_LOWER	R	float(4字节)	读取mit位置下限
0xAA	R_MIT_V_UPPER	R	float(4字节)	读取mit速度上限
0xAB	R_MIT_V_LOWER	R	float(4字节)	读取mit速度下限
0xAC	R_MIT_T_UPPER	R	float(4字节)	读取mit力矩上限
0xAD	R_MIT_T_LOWER	R	float(4字节)	读取mit力矩下限
0xAE	R_MIT_KP_RANGE	R	float(4字节)	读取mit Kp范围（0-范围值）
0xAF	R_MIT_KD_RANGE	R	float(4字节)	读取mit Kd范围（0-范围值）

报警处理命令

命令码	命令名称	方向	数据格式	描述
0xFE	CLEAR_ALARM	W	-	清除报警
0xFD	R_ALARM	R	uint32_t(4字节)	读取报警信息 代码块 1 typedef union {

				<pre>2 struct { 3 uint32_t UNDER_VOLT 4 : 1; 5 uint32_t OVER_VOLT : 6 1; 7 uint32_t 8 OVER_CURRENT : 1; 9 uint32_t 10 OVER_MOTOR_TEMP : 1; 11 uint32_t 12 OVER_CORE_TEMP : 1; 13 uint32_t 14 OVER_INV_TEMP : 1; 15 uint32_t OVER_VEL : 16 1; 17 uint32_t CALIBRATION 18 : 1; 19 uint32_t ENCODER : 1; 20 uint32_t RESERVE : 21 23; 22 }bits; 23 uint32_t value; 24 }MotorErrorUnion;</pre>
--	--	--	--	--

3.3 特殊的标准协议包

3.3.1

MIT控制协议包，非广播包由于CMD占用一个字节，相比广播包数据速度，转矩，Kp，Kd都被压缩为了10比特位。

命令下发

字节	位	描述
0	8位	CMD: 0x0B
1-2	16位	位置指令
3-4	10位(3:0-7,4:0-1)	速度指令
4-5	10位(4:2-7,5:0-3)	Kp参数
5-6	10位(5:4-7,6:0-5)	Kd参数

代码块

```

1 void CMD_SetMIT(uint8_t id , uint32_t length , uint8_t *data) {
2     uint8_t rxData[8] = {0};
3     memcpy(rxData, &data[1], 7);
4     int pInt = (rxData[0]<<8)|rxData[1];
5     int vInt = (rxData[2]<<2)|(rxData[3]>>6);
6     int kpInt = ((rxData[3]&0x3F)<<4)|(rxData[4]>>4);
7     int kdInt = ((rxData[4]&0xF)<<6)|(rxData[5]>>2);
8     int tInt = ((rxData[5]&0x3)<<8)|rxData[6];
9
10    motor.set.mitPos = util_uint2Float(pInt,motor.para.val.mit.posLower,
    motor.para.val.mit.posUpper , 16);
11    motor.set.mitVel = util_uint2Float(vInt,motor.para.val.mit.velLower,
    motor.para.val.mit.velUpper , 10);
12    motor.set.mitTor = util_uint2Float(tInt,motor.para.val.mit.torLower,
    motor.para.val.mit.torUpper , 10);
13    motor.set.mitKp = util_uint2Float(kpInt,0, motor.para.val.mit.kpRange ,
    10);
14    motor.set.mitKd = util_uint2Float(kdInt,0, motor.para.val.mit.kdRange ,
    10);
15
16
17    uint8_t txData[8] = {0};
18    // int posInt =
    util_float2Uint(motor.set.mitPos,motor.para.val.mit.posLower,
    motor.para.val.mit.posUpper , 16);
19    int p_int = util_float2Uint(motor.monitor.pos,
    motor.para.val.mit.posLower, motor.para.val.mit.posUpper, 16);
20    int v_int = util_float2Uint(motor.monitor.vel,
    motor.para.val.mit.velLower, motor.para.val.mit.velUpper, 12);
21    int t_int = util_float2Uint(motor.monitor.tor,
    motor.para.val.mit.torLower, motor.para.val.mit.torUpper, 12);
22    txData[0] = rxData[0]; //cmd
23    txData[1] = p_int>>8;
24    txData[2] = p_int&0xFF;
25    txData[3] = v_int>>4;
26    txData[4] = ((v_int&0xF)<<4) + (t_int>>8);
27    txData[5] = t_int&0xFF;
28    txData[6] = (uint8_t)(motor.error.value >> 8);
29    txData[7] = (uint8_t)motor.error.value;
30    cmdDriver->send_data(id,8, txData);
31 }

```

字节	位	描述
0	8位	CMD:
1-2	16位	位置指令
3-4	12位(2:0-7,3:0-3)	速度指令
4-5	12位(3:4-7,4:0-7)	转矩指令
6	8位	错误信息16: 8位
7	8位	错误信息7: 0位

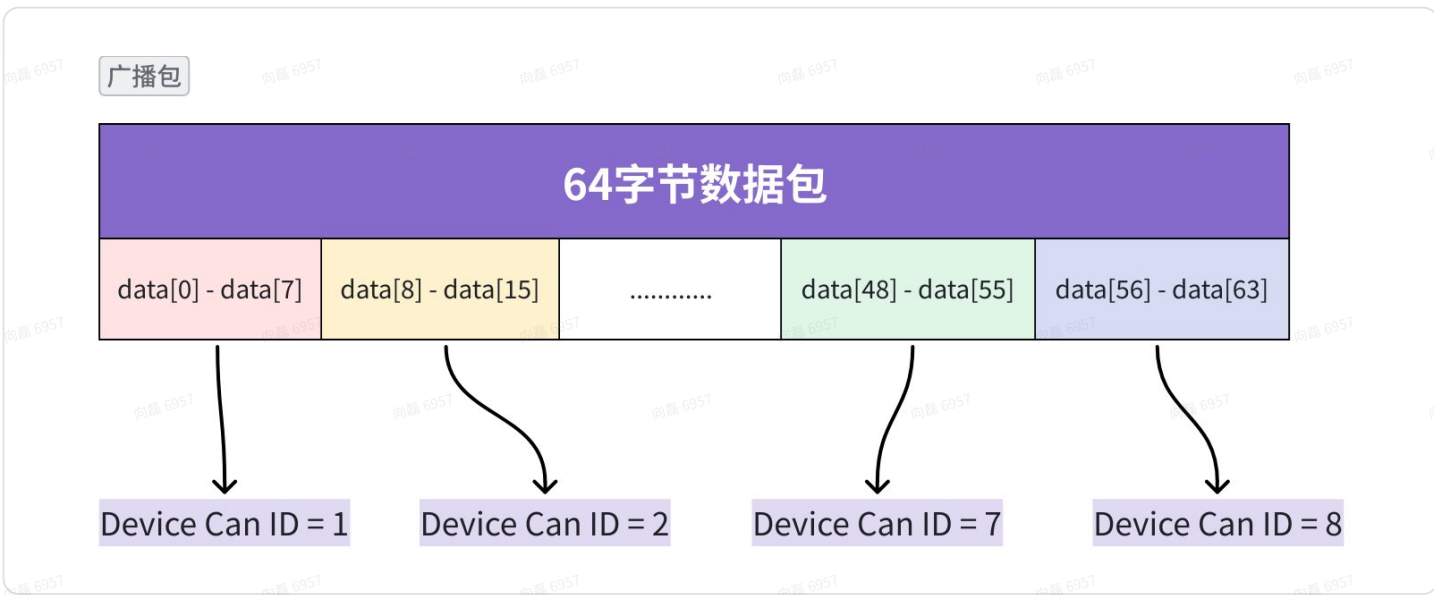
代码块

```
1 void CMD_MIT_Received(uint8_t id,uint8_t *data) {
2     uint8_t txData[8] = {0};
3     // int posInt =
    util_float2Uint(motor.set.mitPos,motor.para.val.mit.posLower,
    motor.para.val.mit.posUpper , 16);
4     int p_int = util_float2Uint(motor.monitor.pos,
    motor.para.val.mit.posLower, motor.para.val.mit.posUpper, 16);
5     int v_int = util_float2Uint(motor.monitor.vel,
    motor.para.val.mit.vellower, motor.para.val.mit.velUpper, 12);
6     int t_int = util_float2Uint(motor.monitor.tor,
    motor.para.val.mit.torLower, motor.para.val.mit.torUpper, 12);
7     txData[0] = data[0]; //cmd
8     txData[1] = p_int>>8;
9     txData[2] = p_int&0xFF;
10    txData[3] = v_int>>4;
11    txData[4] = ((v_int&0xF)<<4) + (t_int>>8);
12    txData[5] = t_int&0xFF;
13    txData[6] = (uint8_t)(motor.error.value >> 8);
14    txData[7] = (uint8_t)motor.error.value;
15    cmdDriver->send_data(id,8, txData);
16 }
```

4. 广播协议包

4.1 命令包格式

命令包主要用于MIT协议发送，ID为1-8的八个设备，按照地址偏移获取广播包内对应的八个字节再加以解析



 广播包代码偏移如下图，如果需要区分ID，不同CAN设备上的ID按照8个ID间隔分布，不影响广播包接收；比如左腿ID：1-7；右腿ID：8-15；

代码块

```
1  uint8_t offset = motor.para.val.canId % 8;
2  memcpy(rxData,(data + (8 * (offset - 1))),8);
```

广播指令抓包：

	时间标识	源通道	帧ID	CAN类型	方向	长度	数据
序号				全部	全部		
0	0.000000	0	0xFF	CANFD加速	Tx	64	01 00 00 00 00 00 00 00 01 00 00 00 00 00 00 01 00 00 00 00 00 00 01 00 00 00 00 00 00 01 00 00 00 00 00 00 01 00 00 00 00 00 00 01 00 00 00 00 00 00 01 00 00 00 00 00 00 ...
1	0.000082	0	0x1	CANFD加速	Rx	5	01 01 07 56 01
2	0.000047	0	0x2	CANFD加速	Rx	5	01 01 07 56 01
3	0.000047	0	0x3	CANFD加速	Rx	5	01 01 07 38 01
4	0.000045	0	0x4	CANFD加速	Rx	5	01 01 07 38 01
5	0.000046	0	0x5	CANFD加速	Rx	5	01 01 07 38 01

4.2 MIT控制命令 (广播ID==0x00)

命令下发

字节	位	描述
0-1	16位	位置指令

2-3	12位(2:0-7,3:0-3)	速度指令
3-4	12位(3:4-7,4:0-7)	Kp参数
5-6	12位(5:0-7,6:0-3)	Kd参数
6-7	12位(6:4-7,7:0-7)	转矩指令

数据上报

字节	位	描述
0-1	16位	位置指令
2-3	12位(2:0-7,3:0-3)	速度指令
3-4	12位(3:4-7,4:0-7)	Kp参数
5	8位	母线电压
6	8位	电机温度
7	8位	错误信息

命令代码

代码块

```
1  int util_float2Uint(float x, float x_min, float x_max, int bits){
2      /// Converts a float to an unsigned int, given range and number of bits ///
3      float span = x_max - x_min;
4      float offset = x_min;
5      return (int) ((x-offset)*((float)((1<<bits)-1))/span);
6  }
7
8  float util_uint2Float(int x_int, float x_min, float x_max, int bits){
9      /// converts unsigned int to float, given range and number of bits ///
10     float span = x_max - x_min;
11     float offset = x_min;
12     return ((float)x_int)*span/((float)((1<<bits)-1)) + offset;
13 }
14
15 void CMD_Receive_MitControl() {
16     uint8_t txData[8] = {0};
17     // int posInt =
18     util_float2Uint(motor.set.mitPos,motor.para.val.mit.posLower,
```

```

motor.para.val.mit.posUpper , 16);
18     int p_int = util_float2Uint(motor.monitor.pos,
motor.para.val.mit.posLower, motor.para.val.mit.posUpper, 16);
19     int v_int = util_float2Uint(motor.monitor.vel,
motor.para.val.mit.velLower, motor.para.val.mit.velUpper, 12);
20     int t_int = util_float2Uint(motor.monitor.tor,
motor.para.val.mit.torLower, motor.para.val.mit.torUpper, 12);
21     txData[0] = p_int>>8;
22     txData[1] = p_int&0xFF;
23     txData[2] = v_int>>4;
24     txData[3] = ((v_int&0xF)<<4) + (t_int>>8);
25     txData[4] = t_int&0xFF;
26     txData[5] = (uint8_t)motor.monitor.vBus;
27     txData[6] = (uint8_t)motor.monitor.motorTemp;
28     txData[7] = (uint8_t)motor.error.value;
29     cmdDriver->send_data(motor.para.val.canId,8, txData);
30 }
31
32 void MIT_CMD_Controller(uint8_t *rxData) {
33     int pInt = (rxData[0]<<8)|rxData[1];
34     int vInt = (rxData[2]<<4)|(rxData[3]>>4);
35     int kpInt = ((rxData[3]&0xF)<<8)|rxData[4];
36     int kdInt = (rxData[5]<<4)|(rxData[6]>>4);
37     int tInt = ((rxData[6]&0xF)<<8)|rxData[7];
38
39     motor.set.mitPos = util_uint2Float(pInt,motor.para.val.mit.posLower,
motor.para.val.mit.posUpper , 16);
40     motor.set.mitVel = util_uint2Float(vInt,motor.para.val.mit.velLower,
motor.para.val.mit.velUpper , 12);
41     motor.set.mitTor = util_uint2Float(tInt,motor.para.val.mit.torLower,
motor.para.val.mit.torUpper , 12);
42     motor.set.mitKp = util_uint2Float(kpInt,0, motor.para.val.mit.kpRange ,
12);
43     motor.set.mitKd = util_uint2Float(kdInt,0, motor.para.val.mit.kdRange ,
12);
44     CMD_Receive_MitControl();
45 }

```

4.3 标准广播包（广播ID==0xFF）

ID为0xFF的时候，根据ID 1到ID 8 偏移，协议解析格式参考标准协议包；

代码块

```

1  if (id == 0xFF) {
2      if (length < 8)

```

```

3         return;
4         uint8_t rxData[64];
5         uint8_t offset = motor.para.val.canId % 8;
6         memcpy(rxData,(data + (8 * (offset - 1))),8);
7         //memcpy(rxData,(data + (8 * (motor.para.val.canId - 1))),8);
8         CmdFunction[rxData[0]](id,length, rxData);
9     }

```

5. 错误信息表

代码块

```

1     typedef union {
2         struct {
3             uint32_t UNDER_VOLT : 1;
4             uint32_t OVER_VOLT : 1;
5             uint32_t OVER_CURRENT : 1;
6             uint32_t OVER_MOTOR_TEMP : 1;
7             uint32_t OVER_CORE_TEMP : 1;
8             uint32_t OVER_INV_TEMP : 1;
9             uint32_t OVER_VEL : 1;
10            uint32_t CALIBRATION : 1;
11            uint32_t ENCODER : 1;
12            uint32_t STEP : 1;
13            uint32_t CAN_ERROR : 1;
14            uint32_t RESERVE : 21;
15        }bits;
16        uint32_t value;
17    }MotorErrorUnion;

```